Zela+ — Automação ESP32 (Sensor de Presença)

Documentação técnica do módulo de hardware que alimenta a detecção de presença/movimento do Zela+. Firmware em `firmware/zela_presenca/zela_presenca.ino`, recebido pelo backend em `POST /ingest/esp32` (`zela/api/ingestao.py`).

1. Visão geral

Um ESP32 lê 4 sensores PIR (infravermelho passivo) — 2 por cômodo, em 2 cômodos — e monitora continuamente se há alguém se movendo. Cada cômodo tem um sensor SR602 e um HC-SR505: dois modelos diferentes no mesmo cômodo, por redundância (reduz falso-negativo caso um dos dois falhe em detectar um movimento específico). O firmware combina os 4 sensores com um OR lógico — presença é considerada "detectada" se qualquer um dos 4 disparar — e só envia uma leitura ao backend quando esse resultado combinado muda de estado (presente ↔ ausente), evitando tráfego desnecessário. Essa leitura alimenta a regra de risco do sistema: se não houver nenhuma presença registrada nas últimas 6 horas, o Zela+ classifica o evento como `RISCO` (`zela/domain/monitoramento.py`) e aciona a escada de escalonamento de alerta para a família.

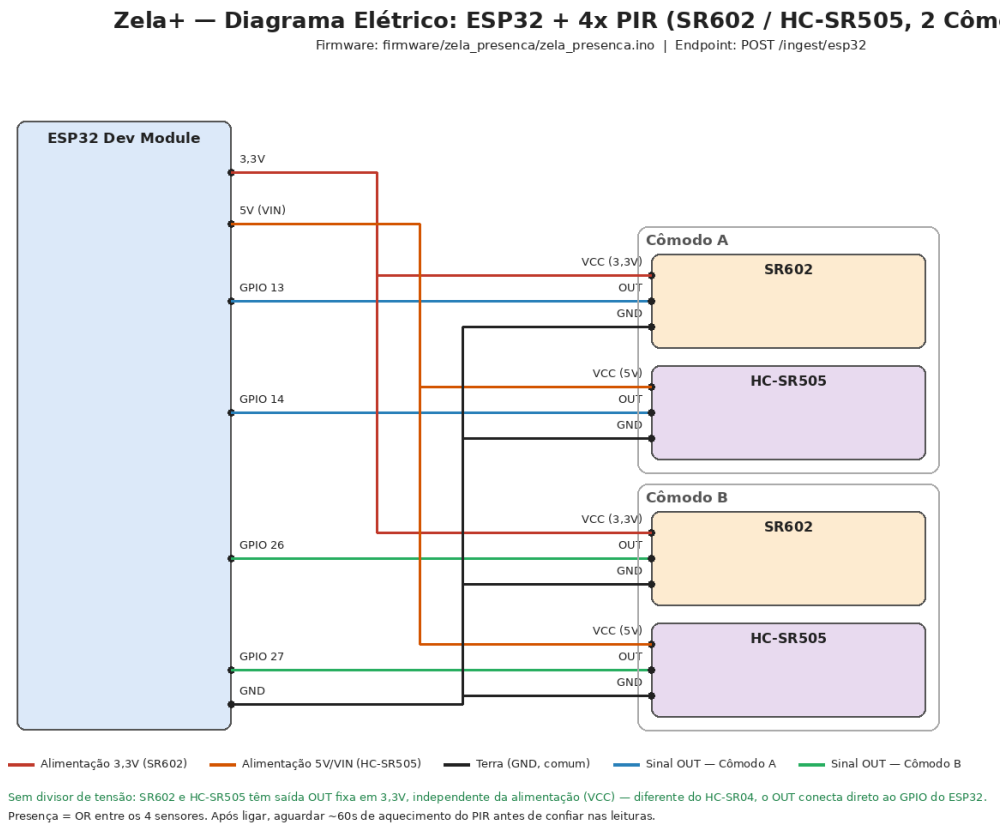
Observação de escopo: com 1 único ESP32 cobrindo os 2 cômodos, o sistema não distingue em qual cômodo houve o movimento — o payload enviado ao backend continua sendo um único booleano de presença, combinado dos 4 sensores. Detecção por cômodo exigiria um ESP32 por cômodo (cada um com sua própria leitura de presença).

2. Lista de materiais (BOM)

Qtd.	Componente	Observação
1	ESP32 Dev Module (ex.: ESP32-WROOM-32 DevKit)	com WiFi integrado
2	Sensor PIR SR602	1 por cômodo; alcance ~5m, alimentação 3,3–15V
2	Sensor PIR HC-SR505	1 por cômodo; alcance ~3m, alimentação 4,5–20V
—	Jumpers macho-fêmea + protoboard	para montagem sem solda; fiação entre cômodos precisa de cabo mais longo
1	Cabo USB (alimentação/gravação)	conforme o conector da placa (micro-USB ou USB-C)

Nenhum resistor é necessário — ver seção 5.

3. Diagrama elétrico



4. Tabela de pinagem

Sensor	Cômodo	VCC	OUT → ESP32	GND
SR602	A	3,3V	GPIO 13	GND
HC-SR505	A	5V (VIN)	GPIO 14	GND
SR602	B	3,3V	GPIO 26	GND
HC-SR505	B	5V (VIN)	GPIO 27	GND

GND é um trilho comum entre o ESP32 e os 4 sensores.

5. Por que não precisa de divisor de tensão (diferente do HC-SR04)

Tanto o SR602 quanto o HC-SR505 têm uma saída `OUT` fixa em ~3,3V, independente da tensão de alimentação (`VCC`) — isso é diferente do HC-SR04 (usado numa versão anterior deste módulo), cujo `ECHO` segue a tensão de alimentação (5V) e exigia um divisor resistivo antes do GPIO. Aqui, `OUT` conecta direto ao GPIO do ESP32 nos dois sensores, sem nenhum componente adicional.

Cada sensor ainda precisa da tensão de alimentação correta: o SR602 aceita 3,3V (pode sair do pino `3,3V` do ESP32); o HC-SR505 pede no mínimo 4,5V segundo o datasheet, por isso é alimentado pelo pino `5V` (`VIN`).

6. Lógica do firmware

Resumo de `firmware/zela_presenca/zela_presenca.ino`:

1. No boot: conecta ao WiFi (timeout de 30s) e sincroniza o relógio via NTP (`pool.ntp.org`) — o ESP32 não tem RTC com bateria própria, então sem isso os timestamps enviados seriam inválidos (época Unix).
2. Aguarda ~60s de aquecimento (`TEMPO_AQUECIMENTO_MS`) antes de confiar em qualquer leitura — sensores PIR costumam oscilar/disparar falso positivo logo após ligar.
3. A cada 2 segundos (`INTERVALO_LEITURA_MS`), lê os 4 pinos digitais (`digitalRead`) e calcula presença = OR entre os 4.
4. Só envia ao backend quando o estado combinado muda (presente ↔ ausente) — não a cada leitura.
5. Se o WiFi cair, tenta reconectar a cada 15s (`INTERVALO_RECONEXAO_MS`) sem bloquear o loop principal.

7. Configuração de rede e segurança

Antes de gravar o firmware, copiar `firmware/zela_presenca/config.h.example` para `firmware/zela_presenca/config.h` (não versionado — evita vazar credenciais no repositório público) e preencher:

Constante	Valor
`WIFI_SSID` / `WIFI_SENHA`	credenciais da rede WiFi local
`URL_INGESTAO`	endereço do backend — rede local (`http://192.168.x.x:8000/ingest/esp32`) ou VPS pública (`https://seu-dominio.com/ingest/esp32`)
`TOKEN_INGESTAO`	mesmo valor de `ZELA_INGESTAO_TOKEN` no backend — obrigatório se o backend estiver exposto fora da rede local (ver `README.md`, seção de configuração)

Quando `TOKEN_INGESTAO` está preenchido, o firmware envia o header `X-Api-Key` em toda requisição; o backend rejeita com `401` qualquer requisição sem o token correto.

8. Fluxo de dados no sistema

```
4x PIR (OR, mudança de estado) -> ESP32 -> POST /ingest/esp32 (X-Api-Key)
 -> LeituraSensor(fonte=ESP32, tipo=PRESENCA) salva no SQLite
 -> job periódico (a cada 15 min, zela/api/scheduler.py)
    -> classificar_por_regra(): >=6h sem presença? -> StatusMonitoramento.RISCO
       -> aplicar_escalonamento() -> alerta enviado à família via WhatsApp
```

9. Checklist de montagem e teste

1. Montar o circuito conforme o diagrama da seção 3 (2 sensores em cada um dos 2 cômodos, fiação de volta até o ESP32).
2. Copiar `config.h.example` para `config.h` e preencher conforme a seção 7.
3. Abrir `firmware/zela_presenca/zela_presenca.ino` no Arduino IDE, placa "ESP32 Dev Module" (pacote "esp32 by Espressif Systems" instalado via Boards Manager).
4. Gravar e abrir o Serial Monitor (115200 bps).
5. Confirmar no log: conexão WiFi, sincronização NTP, e a mensagem de aquecimento dos sensores (~60s).
6. Após o aquecimento, provocar movimento perto de qualquer um dos 4 sensores e confirmar `POST /ingest/esp32 -> 200` no Serial Monitor.
7. Confirmar que a leitura chegou no backend (painel Streamlit ou consulta direta ao `zela.db`).